UNIVERSIDAD NACIONAL DE LUJÁN

Departamento de Ciencias Básicas • Lic. en Sistemas

Programación Orientada a Objetos

(11[0|2]76)

Clase 07 • **Ejercicio 07** • Ciclo Lectivo 2026

MATERIAL PARA ESTUDIANTES
ENUNCIADO PRÁCTICO

Ejercicio 7 (Desafío Integrador): Procesador de Pagos y Pasarelas Financieras

Contexto

En este ejercicio integrador consolidaremos los conceptos fundamentales de la programación orientada a objetos trabajados a lo largo de la materia: la distinción entre **Clases Abstractas** e **Interfaces**, la orquestación mediante el patrón **Template Method**, y los principios de diseño SOLID: **Segregación de Interfaces (ISP)**, **Sustitución de Liskov (LSP)**, **Inversión de Dependencias (DIP)** y **Abierto/Cerrado (OCP)**.

Evolución Conceptual respecto al Ejercicio 2

En el **Ejercicio 2: Tienda de Ropa y Medios de Pago**, modelamos `MedioDePago` como una **interfaz pura** con el método `calcularTotal(double montoBase)`. En aquel problema de dominio, cada medio de pago funcionaba exclusivamente como una función de cálculo matemático de descuento o recargo (patrón Strategy puro), sin identidad persistente, sin marca temporal ni estado mutable a lo largo del tiempo.

Sin embargo, en el mundo real de las pasarelas financieras de comercio electrónico, procesar un cobro trasciende el cálculo aritmético: es una **operación transaccional con ciclo de vida, trazabilidad temporal,**

requisitos de auditoría y restricciones legales. Una transacción posee una identidad unívoca (`idTransaccion`), una marca temporal de ejecución (`fechaHora`), un monto convenido (`monto`), y un estado de confirmación (`exitoso`) que debe preservarse. Por ende, la arquitectura evoluciona naturalmente hacia una **Clase Abstracta** que encapsula el estado común y orquesta el flujo transaccional.

Enunciado

Una plataforma de pagos electrónicos requiere procesar cobros, gestionar devoluciones y emitir auditorías fiscales sobre diversos canales de pago.

1. Jerarquía de Medios de Pago y Ciclo de Vida Transaccional

Diseñe la clase abstracta `MetodoPago` que encapsule el estado compartido por toda transacción financiera:

- Atributos protegidos: `idTransaccion: String` , `monto: double` (debe ser mayor a 0), `fechaHora: LocalDateTime` y `exitoso: boolean` (inicialmente `false`).
- Métodos observadores (getters) públicos para `idTransaccion` , `monto` , `fechaHora` y `isExitoso()` .

Flujo Transaccional Invariable (Patrón Template Method): Toda operación de cobro debe regirse por un ciclo de vida uniforme que previene mover fondos ante datos inválidos:

- Diseñe en `MetodoPago` el método concreto `public boolean procesarPago()` : debe validar la operación mediante un método abstracto `validar()` ; si la validación falla, marca la transacción como fallida (`exitoso = false`) y aborta de inmediato retornando `false` . Si es válida, delega la transferencia en el método abstracto `debitar()` . Si el débito prospera, confirma la operación (`exitoso = true`) y retorna `true` ; de lo contrario, registra el fracaso y retorna `false` .
- Declare los métodos abstractos protegidos:
  - `protected abstract boolean validar()`
  - `protected abstract boolean debitar()`

2. Especialización de Canales Concretos de Pago

Implemente las siguientes pasarelas que extienden de `MetodoPago`, especializando su validación y política de débito:

1. **`TarjetaCredito`:**
    - Atributos privados: `numeroTarjeta: String`, `cvv: String`, `cantCuotas: int` (≥ 1), `limiteDisponible: double`.
    - `validar()`: Comprueba que `numeroTarjeta` posea exactamente 16 dígitos, `cvv` 3 dígitos y `cantCuotas >= 1`.
    - `debitar()`: En 1 cuota no aplica recargo. En 2 o más cuotas, aplica un recargo del 2.5% adicional por cada cuota a partir de la segunda (ej: 3 cuotas = 5.0%; 4 cuotas = 7.5%). Si el monto final con recargo no excede el `limiteDisponible`, descuenta el importe del límite y confirma el débito (`true`); de lo contrario, lo rechaza (`false`).
    - Getter para `getLimiteDisponible()`.
2. **`TransferenciaBancaria`:**
    - Atributos privados: `cbu: String`, `bancoOrigen: String`.
    - `validar()`: Comprueba que el CBU posea exactamente 22 dígitos numéricos.
    - `debitar()`: Simula la confirmación interbancaria en tiempo real retornando `true`.
3. **`Criptomoneda`:**
    - Atributos privados: `direccionBilletera: String`, `redBlockchain: String`, `gasFee: double` (comisión de red).
    - `validar()`: Comprueba que la dirección comience con el prefijo `"0x"` y no esté vacía.
    - `debitar()`: Confirma la transferencia en blockchain (`true`) únicamente si el monto de la transacción es estrictamente superior a la comisión de red (`gasFee`); caso contrario, rechaza la operación (`false`).
    - Getter para `getGasFee()`.

3. Desafío de Diseño: Capacidades Segregadas y Restricciones del Dominio (ISP y LSP)

En el comercio internacional, no todos los medios de pago admiten las mismas operaciones operativas y legales:

- **Capacidad de Reembolso:** Ciertos medios admiten la restitución de importes (`boolean procesarReembolso(double montoReembolso)`). Una tarjeta puede reintegrar saldo a su límite disponible y un banco simular un contracargo. **Sin embargo, las transacciones en blockchain son por definición inmutables e irreversibles**: una criptomoneda jamás puede ser reembolsada.
    - *Principio de Segregación de Interfaces (ISP):* Diseñe la interfaz `Reembolsable`. Evite forzar este método en la superclase `MetodoPago` para no obligar a `Criptomoneda` a violar el Principio de Sustitución de Liskov (LSP) lanzando excepciones en tiempo de ejecución. Aplique el contrato exclusivamente a los canales que admitan reintegros.
- **Capacidad de Auditoría Fiscal:** Los canales bancarizados tradicionales deben emitir comprobantes impositivos (`String generarComprobanteFiscal()`) detallando ID de transacción, monto gravado, 21% de IVA y el medio interviniente.
    - Diseñe la interfaz `AuditableFiscalmente` y asígnela a los canales sujetos a regulación fiscal tradicional.

4. Orquestador Central (`ProcesadorVentas`)

Diseñe el componente central `ProcesadorVentas` que procese cobros, devoluciones y auditorías operando exclusivamente contra abstracciones:

- `public boolean procesarCobro(MetodoPago pago)` : Ejecuta el cobro aprovechando polimórficamente el Template Method.
- `public boolean gestionarDevolucion(Reembolsable medioReembolsable, double monto)` : Procesa devoluciones blindadas por el sistema de tipos de Java, asegurando en tiempo de compilación que sea imposible enviar un medio irreversible como `Criptomoneda` .
- `public String auditarOperacion(AuditableFiscalmente auditable)` : Emite el comprobante legal delegando en el contrato de auditoría.

Tareas:

1. Diseñar la clase abstracta `MetodoPago` modelando el estado transaccional, los ganchos abstractos de validación/débito y el algoritmo orquestador `procesarPago()` mediante el patrón Template Method.
2. Identificar y diseñar las interfaces de capacidad segregadas `Reembolsable` y `AuditableFiscalmente`.
3. Implementar las clases concretas `TarjetaCredito`, `TransferenciaBancaria` y `Criptomoneda`, deduciendo y aplicando a cada una las capacidades de rol compatibles según las restricciones operativas y normativas del dominio.
4. Implementar la clase `ProcesadorVentas` asegurando desacoplamiento total (DIP y OCP) y seguridad estática de tipos.

Restricciones y Tips

- **Herencia vs Composición de Capacidades:** Note que el estado transaccional común (`idTransaccion`, `monto`, `fechaHora`, `exitoso`) pertenece a la clase abstracta `MetodoPago`, mientras que las capacidades operativas no homogéneas (`Reembolsable`, `AuditableFiscalmente`) se segregan en interfaces.
- **Seguridad de Tipos en Tiempo de Compilación:** Al tipear el parámetro de `gestionarDevolucion` como `Reembolsable`, el compilador de Java impide que una instancia de `Criptomoneda` sea enviada como argumento, evitando caídas en tiempo de ejecución (`UnsupportedOperationException`).
- **Invariantes del Template Method:** En `procesarPago()`, asegúrese de que bajo ninguna circunstancia se invoque a `debitar()` si `validar()` retornó `false`.

Preguntas Conceptuales

Al finalizar el ejercicio, reflexione sobre las siguientes preguntas:

1. ¿Por qué en el Ejercicio 2 `MedioDePago` se modeló como una interfaz pura y en este ejercicio como una clase abstracta? ¿Qué cambio

fundamental en el modelo del dominio impulsó esta decisión arquitectónica?

2. ¿Qué principio de diseño se violaría si hubiésemos colocado el método `procesarReembolso()` dentro de la clase abstracta `MetodoPago`? ¿Qué consecuencias traería esto para la clase `Criptomoneda`?
3. ¿Cómo asegura el patrón **Template Method** en `procesarPago()` que todas las pasarelas cumplan con el ciclo de vida transaccional sin duplicar lógica de control en cada subclase?
4. ¿Por qué `ProcesadorVentas` no necesita utilizar sentencias `if`, `switch` ni el operador `instanceof` para procesar cobros, devoluciones o comprobantes fiscales?